

Project 4: Distance-Vector Routing

This project requires you to implement the distance-vector routing protocol for a distributed set of simulated network entities. Each entity will send and receive packets in accordance with the distance-vector protocol until it has calculated minimum cost routes to all other nodes in the network.

Overview

You will implement the algorithm that runs on each entity in the network. Since with DV all entities run the same algorithm, you will only implement this once and that entity will be instantiated multiple times to form all of the nodes in the network.

The entities will be numbered from 0 to `number_of_entities - 1`, and when each entity is created it will be given its index. The entity will also be given the total number of nodes (including itself) in the network. This will allow each entity to create a data structure for holding the best known costs to all other nodes in the network.

When the simulation begins, each entity will be given the initial costs from itself to each of its neighbors. Since not all entities are connected to all other entities, this will only contain a few edges, and only the edges connected to the current entity.

The network simulator will handle sending packets between the various entities. The simulator will check that the packet is valid (i.e. the source address matches the sender) and that the link layer (layer 2) can handle it (i.e. the packet is being sent between one-hop neighbors).

Each entity in the network should only send information to its neighbors if it has updated information to send. Once all of the lowest cost routes have been determined there should be no more packets in the simulator and the simulation will end.

Note that this implementation uses triggered updates only — entities send packets in response to routing changes, not on a fixed periodic schedule. This differs from real-world DV protocols such as RIP, which also send periodic updates for liveness. Here, the simulation ends naturally once all routing information has propagated and the packet queue is empty.

The network simulator can help with displaying forwarding tables for each entity and simulating packets being routed through the network.

A top-level file will initialize and start the simulation. This includes defining the network (how many entities, which links exist, and the cost of each link).

Software Structure

There are four source files in this project: `project` , `entity` , `packet` , and `network_simulator` .

- `project` : Top-level file that sets up the network and simulation. After the simulation has completed (there are no more packets to send and the route costs have been minimized) this can also simulate packets traversing the network.

The provided main project file provides example networks and an example setup for executing the simulator.

- `entity` : This file is where you will implement the distance-vector algorithm. All of the stub functions you need are already defined, you just need to provide implementations.
- `packet` : A very simple definition of a packet used in the network simulator. You will create these to send packets from each entity.
- `network_simulator` : The actual simulator. You shouldn't need to change this, and an original copy will be used when grading.

Software Interfaces

This main portion of this project is implementing the functions for the entity.

`entity`

There are five functions you must implement for entity:

- `init(index, number_of_entities)` : This function will be called when the entity is created. It should be used to setup data structures.
- `[packets] initialize_costs(neighbor_costs)` : This function will be called once all of the entities have been created to provide the entity with knowledge about its neighbors, and the link costs to those neighbors.

`neighbor_costs` is an array of `(neighbor_index, cost)` tuples. The first value in the tuple is the entity ID (i.e. the entity's index number) of the neighbor, and the second value is the link

cost of the link to that neighbor. Both the neighbor index and cost are integers. For example, if an entity's `initialize_costs()` method is called with: `[(3, 4), (7, 10), (8, 1)]` then that entity has three neighbors with IDs 3, 7, and 8, and the costs to each neighbor are 4, 10, and 1, respectively.

The `initialize_costs()` function should return an array of 0 or more packets to send from this entity. The packets should be of type `Packet`, and have their destination IDs set to the correct destination. The source address can be set or left blank; the simulator will set it to the correct sending entity.

- `[packets] update(packet)` : This function will be called when a packet arrives for the entity. The function will be passed a single packet of type `Packet`.

The return value is the same as `initialize_costs()` : an array of 0 or more packets to send from this entity.

- `[costs] get_all_costs()` : This function should return the forwarding table based on the entity's current best information. Likely this will be used after the simulation is complete to view or use the forwarding table generated from the distance-vector algorithm.

The `get_all_costs()` function should return an array with the same length as the number of entities in the network. So if the network has four nodes then the return array would be of length four. Each element in the array should be a two element tuple `(next_hop, cost)` specifying the forwarding information for each entity in the network (in the order of entity IDs). For example, for a four node network, entity with ID=2 might return: `[(3, 5), (1, 7), (2, 0), (3, 4)]`, meaning that the best route it knows of to get to entity ID=0 is to first send to entity ID=3, and that the total route cost is 5.

- `next_hop forward_next_hop(destination)` : This function is used to see the hops a packet would take to actually traverse a network. Basically, if a packet arrived at this entity destined for the given destination, where should this entity send it? That is, which neighbor should it forward it to?

This function should return the entity index of the next hop for this packet that uses the lowest cost route.

Example

Consider the following network as an example:



There are four entities (0-3) with link costs as shown.

When the simulator starts, it will create each entity by calling the `init` function on each entity (in pseudocode):

```
entity_zero = new Entity(0, 4);
entity_one  = new Entity(1, 4);
entity_two  = new Entity(2, 4);
entity_three = new Entity(3, 4);
```

After the entities are constructed it will call `initialize_costs()` for each entity. Using entity ID=3 as an example:

```
neighbor_costs = [(0, 7), (2, 2)];
ret = entity_three.initialize_costs(neighbor_costs);

# `ret` might be an array of two packets:
# [ Packet(destination=0, costs=[7, 999, 2, 0]),
#   Packet(destination=2, costs=[7, 999, 2, 0]) ]
```

Of course it will do this for all entities. Each entity should return any packets it needs to send. The simulator will pass these to the appropriate entities.

At the end, the main project may call `get_all_costs()`. For example, with entity ID=2:

```
all_costs = entity_two.get_all_costs();
```

```
# all_costs should be:
# [(1, 2), (1, 1), (2, 0), (3, 2)]
```

Then if we want to simulate a packet going from entity 2 to 0:

```
next_hop = entity_two.forward_next_hop(0); # should return 1
next_hop = entity_one.forward_next_hop(0); # should return 0
```

packet

The packet interface is relatively straightforward. Note, these packets are sent on layer 2 and therefore can only go single hops between neighboring nodes.

- `init(destination, costs)` : Call the constructor to create a packet. The `destination` is the entity index of the node to send to. `costs` is an array with length equal to the number of entities in the network, and each element is the current known minimum cost to that entity in the network.

`Packet` also supports various getters and setters.

network_simulator

The example `project` file should provide enough information on how to use the simulator.

The `display_forwarding_table()` function is useful to check that the forwarding tables generated by your distance-vector algorithm are correct. The function will print tables that look like:

E0	Cost	Next Hop
0	0	0
1	1	1
2	2	1
3	4	1

This example table happens to be for entity 0, as noted in the top left. The three columns are: the destination entity, the total cost to send a packet to that destination from the given entity, and which hop that entity will use to forward a packet to achieve the listed cost. For example, if entity

ID=0 wants to send a packet to entity ID=2, the total cost of the trip will be 2 and entity ID=0 will forward the packet to its neighbor entity ID=1.

Testing

You should use the example networks provided in `project.py` to test your implementation. Compare the resulting forwarding tables to tables you generate manually to verify correctness. You should also use `route_packet()` to simulate packets traversing the network and confirm they follow the path you expect.

Submission

You must submit your completed `entity.py` file.

Helpful Hints and the Like

- A debugging value of 2 may be helpful to you in debugging your code. You should keep in mind that real implementers do not have underlying networks that provide such nice information about what is going to happen to their packets!
- You can initialize the costs in the data structure(s) in your entity implementations to 999.
- Your entity may be tested on networks that contain disconnected components. Nodes in a separate component are unreachable; their cost should remain at the initial value.
- I suggest reviewing the Bellman-Ford equation from lecture, and making sure you understand how to calculate updated path costs before trying to implement the code.